

```

import React, { useState, useEffect } from 'react';
import { ChevronLeft, ChevronRight, Plus, Trash2, Users, Camera } from 'lucide-react';

export default function FitnessDashboard() {
  const [activeTab, setActiveTab] = useState('dashboard');
  const [currentMonth, setCurrentMonth] = useState(new Date());
  const [selectedDate, setSelectedDate] = useState(new Date().toISOString().split('T')[0]);
  const [entries, setEntries] = useState({});
  const [profiles, setProfiles] = useState([]);
  const [currentProfile, setCurrentProfile] = useState(null);
  const [showProfileModal, setShowProfileModal] = useState(false);
  const [newProfileName, setNewProfileName] = useState('');

  // Meal tracking
  const [selectedFood, setSelectedFood] = useState('');
  const [mealType, setMealType] = useState('breakfast');
  const [showManualMeal, setShowManualMeal] = useState(false);
  const [manualMeal, setManualMeal] = useState({ name: '', calories: '' });

  // Workout tracking
  const [selectedWorkout, setSelectedWorkout] = useState('');

  // Supplement tracking
  const [selectedSupplement, setSelectedSupplement] = useState('');

  // Alcohol tracking
  const [selectedAlcohol, setSelectedAlcohol] = useState('');
  const [alcoholQty, setAlcoholQty] = useState('1');

  // Progress pictures
  const [progressPictures, setProgressPictures] = useState({});

  // Initialize profiles
  useEffect(() => {
    const saved = localStorage.getItem('fitnesProfiles');
    if (saved) {
      const profilesList = JSON.parse(saved);
      setProfiles(profilesList);
      setCurrentProfile(profilesList[0]?.id || null);
    } else {
      const newProfile = { id: 'p1', name: 'My Profile' };
      setProfiles([newProfile]);
      setCurrentProfile('p1');
      localStorage.setItem('fitnesProfiles', JSON.stringify([newProfile]));
    }
  }, []);
}

```

```

    }
  }, []);

// Load profile data
useEffect(() => {
  if (!currentProfile) return;
  const key = `entries_${currentProfile}`;
  const saved = localStorage.getItem(key);
  if (saved) {
    setEntries(JSON.parse(saved));
  }
}, [currentProfile]);

// Save profile data
useEffect(() => {
  if (!currentProfile) return;
  const key = `entries_${currentProfile}`;
  localStorage.setItem(key, JSON.stringify(entries));
}, [entries, currentProfile]);

// Ensure current date has entry
useEffect(() => {
  if (!entries[selectedDate]) {
    setEntries(prev => ({
      ...prev,
      [selectedDate]: { meals: {}, workouts: [], supplements: [], alcohol: [] }
    }));
  }
}, [selectedDate, entries]);

// Databases
const foods = [
  { name: 'Chicken Breast', calories: 165, protein: 31, carbs: 0, fat: 3.6 },
  { name: 'Brown Rice', calories: 215, protein: 5, carbs: 45, fat: 1.8 },
  { name: 'Salmon', calories: 208, protein: 20, carbs: 0, fat: 13 },
  { name: 'Broccoli', calories: 55, protein: 4, carbs: 11, fat: 0.6 },
  { name: 'Banana', calories: 105, protein: 1.3, carbs: 27, fat: 0.3 },
  { name: 'Oatmeal', calories: 150, protein: 5, carbs: 27, fat: 3 },
  { name: 'Egg', calories: 78, protein: 6, carbs: 0.6, fat: 5.3 },
  { name: 'Greek Yogurt', calories: 59, protein: 10, carbs: 3.3, fat: 0.4 },
  { name: 'Almonds', calories: 164, protein: 6, carbs: 6, fat: 14 },
  { name: 'Pasta', calories: 220, protein: 8, carbs: 43, fat: 1.1 },
];

const workouts = [
  { name: 'Running', calories: 300, icon: '🏃' },
  { name: 'Cycling', calories: 400, icon: '🚴' },

```

```

    { name: 'Weight Training', calories: 250, icon: '🏋️' },
    { name: 'Swimming', calories: 280, icon: '🏊' },
    { name: 'Yoga', calories: 180, icon: '🧘' },
    { name: 'Walking', calories: 120, icon: '🚶' },
    { name: 'Basketball', calories: 450, icon: '🏀' },
  ];

const supplements = [
  { name: 'Vitamin D', dosage: '1000 IU', icon: '☀️', category: 'Vitamins' },
  { name: 'Vitamin C', dosage: '500mg', icon: '🍊', category: 'Vitamins' },
  { name: 'Magnesium', dosage: '400mg', icon: '⚡️', category: 'Minerals' },
  { name: 'Zinc', dosage: '15mg', icon: '🛡️', category: 'Minerals' },
  { name: 'Testosterone Booster', dosage: '500mg', icon: '💪', category: 'Perfc' },
  { name: 'Omega-3', dosage: '2000mg', icon: '🐟', category: 'Omega' },
  { name: 'Whey Protein', dosage: '25g', icon: '🥛', category: 'Proteins' },
];

const alcohols = [
  { name: 'Beer', calories: 150, icon: '🍺', abv: '5%' },
  { name: 'Wine', calories: 120, icon: '🍷', abv: '12%' },
  { name: 'Whiskey', calories: 100, icon: '🥃', abv: '40%' },
  { name: 'Vodka', calories: 97, icon: '🍸', abv: '40%' },
  { name: 'Gin', calories: 110, icon: '🍹', abv: '40%' },
  { name: 'Margarita', calories: 170, icon: '🍹', abv: '25%' },
  { name: 'Mojito', calories: 140, icon: '🍃', abv: '15%' },
];

// Current data
const currentData = entries[selectedDate] || { meals: {}, workouts: [], supplem
const mealTypes = ['breakfast', 'lunch', 'dinner', 'snack'];
const totalCaloriesIn = Object.values(currentData.meals || {}).flat().reduce((s
const totalCaloriesBurned = (currentData.workouts || []).reduce((sum, w) => sum
const totalAlcoholCalories = (currentData.alcohol || []).reduce((sum, a) => sum

// Functions
const addMeal = () => {
  const food = foods.find(f => f.name === selectedFood);
  if (!food) return;
  setEntries(prev => ({
    ...prev,
    [selectedDate]: {
      ...prev[selectedDate],
      meals: {
        ...prev[selectedDate].meals,
        [mealType]: [...(prev[selectedDate].meals[mealType] || []), { ...food,
      }
    }
  })
}

```

```

    ));
    setSelectedFood('');
  };

const addManualMeal = () => {
  if (!manualMeal.name || !manualMeal.calories) return;
  setEntries(prev => ({
    ...prev,
    [selectedDate]: {
      ...prev[selectedDate],
      meals: {
        ...prev[selectedDate].meals,
        [mealType]: [...(prev[selectedDate].meals[mealType] || []), { name: man
      }
    }
  }));
  setManualMeal({ name: '', calories: '' });
  setShowManualMeal(false);
};

const removeMeal = (type, id) => {
  setEntries(prev => ({
    ...prev,
    [selectedDate]: {
      ...prev[selectedDate],
      meals: {
        ...prev[selectedDate].meals,
        [type]: prev[selectedDate].meals[type].filter(m => m.id !== id)
      }
    }
  }));
};

const addWorkout = () => {
  const workout = workouts.find(w => w.name === selectedWorkout);
  if (!workout) return;
  setEntries(prev => ({
    ...prev,
    [selectedDate]: {
      ...prev[selectedDate],
      workouts: [...(prev[selectedDate].workouts || []), { ...workout, id: Date
    }
  }));
  setSelectedWorkout('');
};

const removeWorkout = (id) => {

```

```

    setEntries(prev => ({
      ...prev,
      [selectedDate]: {
        ...prev[selectedDate],
        workouts: (prev[selectedDate].workouts || []).filter(w => w.id !== id)
      }
    }));
  });
};

const addSupplement = () => {
  const supp = supplements.find(s => s.name === selectedSupplement);
  if (!supp) return;
  setEntries(prev => ({
    ...prev,
    [selectedDate]: {
      ...prev[selectedDate],
      supplements: [...(prev[selectedDate].supplements || []), { ...supp, id: D
    }
  }));
  setSelectedSupplement('');
};

const removeSupplement = (id) => {
  setEntries(prev => ({
    ...prev,
    [selectedDate]: {
      ...prev[selectedDate],
      supplements: (prev[selectedDate].supplements || []).filter(s => s.id !==
    }
  }));
};

const addAlcohol = () => {
  const drink = alcohols.find(a => a.name === selectedAlcohol);
  if (!drink) return;
  const qty = parseInt(alcoholQty) || 1;
  setEntries(prev => ({
    ...prev,
    [selectedDate]: {
      ...prev[selectedDate],
      alcohol: [...(prev[selectedDate].alcohol || []), { ...drink, qty, totalCa
    }
  }));
  setSelectedAlcohol('');
  setAlcoholQty('1');
};

```

```

const removeAlcohol = (id) => {
  setEntries(prev => ({
    ...prev,
    [selectedDate]: {
      ...prev[selectedDate],
      alcohol: (prev[selectedDate].alcohol || []).filter(a => a.id !== id)
    }
  }));
};

const addProfile = () => {
  if (!newProfileName.trim()) return;
  const newP = { id: `p${Date.now()}`, name: newProfileName };
  const updated = [...profiles, newP];
  setProfiles(updated);
  setCurrentProfile(newP.id);
  setNewProfileName('');
  setShowProfileModal(false);
  localStorage.setItem('fitnesProfiles', JSON.stringify(updated));
};

const deleteProfile = (id) => {
  if (profiles.length === 1) return;
  const updated = profiles.filter(p => p.id !== id);
  setProfiles(updated);
  setCurrentProfile(updated[0]?.id);
  localStorage.setItem('fitnesProfiles', JSON.stringify(updated));
};

const getDaysInMonth = (date) => new Date(date.getFullYear(), date.getMonth() + 1, 0).getDate();
const getFirstDay = (date) => new Date(date.getFullYear(), date.getMonth(), 1).getDay();
const currentProfileObj = profiles.find(p => p.id === currentProfile);

return (
  <div style={{ background: 'linear-gradient(135deg, #1a1f2e 0%, #222d3d 50%, #2c3e50 50%, #2c3e50 100%)', color: 'white', padding: '20px', border-radius: '10px', width: '100%', margin: '0 auto' }}>
    <style>{`
      @import url('https://fonts.googleapis.com/css2?family=Outfit:wght@700&family=Poppins,sans-serif');
      body { font-family: Poppins, sans-serif; }
      h1, h2, h3 { font-family: Outfit, sans-serif; font-weight: 700; }
      .glass { background: rgba(20, 25, 35, 0.7); border: 1px solid rgba(112, 112, 112, 0.5); }
      .btn { background: linear-gradient(135deg, #70bb8e 0%, #5aa876 100%); color: white; padding: 10px 20px; border-radius: 5px; text-decoration: none; }
      .btn:hover { transform: translateY(-2px); box-shadow: 0 8px 16px rgba(112, 112, 112, 0.5); }
      input, select { background: rgba(40, 50, 70, 0.8); color: #e2e8f0; border: 1px solid #70bb8e; padding: 5px; width: 100%; }
      input:focus, select:focus { outline: none; border-color: #70bb8e; }
    `}</style>

    <div style={{ maxWidth: '1200px', margin: '0 auto' }}>

```

[illegible]

```

        <div style={{ fontSize: '12px', color: '#94a3b8' }}>Calories
        <div style={{ fontSize: '24px', color: '#70bb8e', fontWeight:
</div>
<div style={{ background: 'rgba(112, 187, 142, 0.1)', padding:
    <div style={{ fontSize: '12px', color: '#94a3b8' }}>Burned</d
    <div style={{ fontSize: '24px', color: '#70bb8e', fontWeight:
</div>
</div>
</div>
</div>

<div className="glass" style={{ marginBottom: '16px' }}>
  <h3 style={{ marginTop: 0 }}><img alt="Apple Health icon" data-bbox="555 258 570 273"/> Apple Health</h3>
  <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', ga
    <div style={{ background: 'rgba(112, 187, 142, 0.1)', padding:
      <div style={{ fontSize: '12px', color: '#94a3b8' }}>Steps</di
      <div style={{ fontSize: '20px', color: '#70bb8e', fontWeight:
</div>
    <div style={{ background: 'rgba(112, 187, 142, 0.1)', padding:
      <div style={{ fontSize: '12px', color: '#94a3b8' }}>Heart</di
      <div style={{ fontSize: '20px', color: '#70bb8e', fontWeight:
</div>
    </div>
  </div>
</div>

<div className="glass">
  <h3 style={{ marginTop: 0 }}><img alt="Alcohol Tracking icon" data-bbox="550 525 565 540"/> Alcohol Tracking</h3>
  <div style={{ display: 'grid', gap: '8px', marginBottom: '12px' }
    <select value={selectedAlcohol} onChange={(e) => setSelectedAlc
      <option value="">Select drink</option>
      {alcohols.map(a => <option key={a.name} value={a.name}>{a.ico
    </select>
    <div style={{ display: 'grid', gridTemplateColumns: '1fr auto',
      <input type="number" min="1" max="20" value={alcoholQty} onCh
      <span style={{ padding: '8px 12px', background: 'rgba(40, 50,
    </div>
    <button onClick={addAlcohol} className="btn" style={{ width: '1
  </div>
  <div style={{ background: 'rgba(220, 38, 38, 0.1)', padding: '10p
    <div style={{ fontSize: '12px', color: '#f87171' }}>Total Alcoh
    <div style={{ fontSize: '20px', color: '#dc2626', fontWeight: '
  </div>
  {(currentData.alcohol || []).map(a => (
    <div key={a.id} style={{ display: 'flex', justifyContent: 'spac
      <div>
        <div style={{ color: '#e2e8f0', fontWeight: '500' }}>{a.ico
        <div style={{ fontSize: '12px', color: '#94a3b8' }}>x{a.qty
      </div>

```



```

        <button onClick={() => removeAlcohol(a.id)} style={{ backgrou
    </div>
    )))
  </div>
</div>

{/* Right */}
<div>
  <div className="glass" style={{ marginBottom: '16px' }}>
    <h3 style={{ marginTop: 0 }}><img alt="Meal icon" data-bbox="550 218 570 235"/> Meals</h3>
    <div style={{ display: 'grid', gap: '8px', marginBottom: '12px' }}
      <select value={mealType} onChange={(e) => setMealType(e.target.
        {mealTypes.map(t => <option key={t} value={t}>{t}</option>)}
      </select>
      <select value={selectedFood} onChange={(e) => setSelectedFood(e
        <option value="">Select food</option>
        {foods.map(f => <option key={f.name} value={f.name}>{f.name}<
      </select>
      <button onClick={addMeal} className="btn" style={{ width: '100%
      <button onClick={() => setShowManualMeal(!showManualMeal)} styl
    </div>
    {showManualMeal && (
      <div style={{ padding: '8px', background: 'rgba(112, 187, 142,
        <input type="text" placeholder="Name" value={manualMeal.name}
        <input type="number" placeholder="Calories" value={manualMeal
        <button onClick={addManualMeal} className="btn" style={{ widt
      </div>
    )}
    {mealTypes.map(type => (currentData.meals[type] || []).length > 0
      <div key={type}>
        <p style={{ color: '#70bb8e', fontSize: '12px', fontWeight: '
          {(currentData.meals[type] || []).map(m => (
            <div key={m.id} style={{ display: 'flex', justifyContent: '
              <div>
                <div style={{ color: '#e2e8f0', fontWeight: '500' }}>{m
                <div style={{ fontSize: '12px', color: '#94a3b8' }}>{m.
              </div>
              <button onClick={() => removeMeal(type, m.id)} style={{ b
            </div>
          )}}
        </div>
      )}}
    </div>
  )}}
</div>

<div className="glass" style={{ marginBottom: '16px' }}>
  <h3 style={{ marginTop: 0 }}><img alt="Workout icon" data-bbox="550 908 570 925"/> Workouts</h3>
  <div style={{ display: 'grid', gap: '8px', marginBottom: '12px' }}

```

```

        <select value={selectedWorkout} onChange={ (e) => setSelectedWor
          <option value="">Select workout</option>
          {workouts.map(w => <option key={w.name} value={w.name}>>{w.ico
        </select>
        <button onClick={addWorkout} className="btn" style={{ width: '1
      </div>
    {(currentData.workouts || []).map(w => (
      <div key={w.id} style={{ display: 'flex', justifyContent: 'spac
        <div>
          <div style={{ color: '#e2e8f0', fontWeight: '500' }}>{w.ico
          <div style={{ fontSize: '12px', color: '#94a3b8' }}>-{w.cal
        </div>
        <button onClick={() => removeWorkout(w.id)} style={{ backgrou
      </div>
    )))
  </div>

  <div className="glass">
    <h3 style={{ marginTop: 0 }}><img alt="pill icon" data-bbox="548 391 568 406"/> Supplements</h3>
    <div style={{ display: 'grid', gap: '8px', marginBottom: '12px' }
      <select value={selectedSupplement} onChange={ (e) => setSelected
        <option value="">Select supplement</option>
        {supplements.map(s => <option key={s.name} value={s.name}>>{s.
      </select>
      <button onClick={addSupplement} className="btn" style={{ width:
    </div>
    {(currentData.supplements || []).map(s => (
      <div key={s.id} style={{ display: 'flex', justifyContent: 'spac
        <div>
          <div style={{ color: '#e2e8f0', fontWeight: '500' }}>{s.ico
          <div style={{ fontSize: '12px', color: '#94a3b8' }}>{s.dosa
        </div>
        <button onClick={() => removeSupplement(s.id)} style={{ backg
      </div>
    )))
  </div>
</div>
</div>
)}

{/* CALENDAR */}
{activeTab === 'calendar' && (
  <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '20
    <div className="glass">
      <div style={{ display: 'flex', justifyContent: 'space-between', ali
        <h3 style={{ margin: 0 }}>{currentMonth.toLocaleString('default',
        <div style={{ display: 'flex', gap: '8px' }}>

```

```

        <button onClick={() => setCurrentMonth(new Date(currentMonth.ge
        <button onClick={() => setCurrentMonth(new Date(currentMonth.ge
    </div>
</div>
<div style={{ display: 'grid', gridTemplateColumns: 'repeat(7, 1fr)
    [['S', 'M', 'T', 'W', 'T', 'F', 'S']].map(d => <div key={d} style=
</div>
<div style={{ display: 'grid', gridTemplateColumns: 'repeat(7, 1fr)
    {Array.from({ length: getFirstDay(currentMonth) + getDaysInMonth(
        const dayNum = i - getFirstDay(currentMonth) + 1;
        if (dayNum < 1 || dayNum > getDaysInMonth(currentMonth)) return
        const dateStr = `${currentMonth.getFullYear()}-${String(current
        const hasWorkout = (entries[dateStr]?.workouts || []).length >
        const hasMeal = Object.values(entries[dateStr]?.meals || {}).fl
        const isSelected = selectedDate === dateStr;
        return (
            <button
                key={i}
                onClick={() => setSelectedDate(dateStr)}
                style={{
                    padding: '8px',
                    border: isSelected ? '2px solid #70bb8e' : '1px solid rgb
                    background: isSelected ? 'rgba(112, 187, 142, 0.2)' : has
                    borderRadius: '8px',
                    cursor: 'pointer',
                    color: '#e2e8f0',
                    fontWeight: '600',
                    textAlign: 'center',
                    position: 'relative'
                }}
            >
                {dayNum}
                {hasWorkout && <div style={{ fontSize: '10px', marginTop: '
                {hasMeal && !hasWorkout && <div style={{ fontSize: '10px',
            </button>
        );
    }}}
</div>
</div>

<div className="glass">
    <h3 style={{ marginTop: 0 }}><img alt="calendar icon" data-bbox="532 833 552 848" /> {new Date(selectedDate).toLocaleDat
    <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap:
        <div style={{ background: 'rgba(112, 187, 142, 0.1)', padding: '1
            <div style={{ fontSize: '11px', color: '#94a3b8' }}>Calories</d
            <div style={{ fontSize: '20px', color: '#70bb8e', fontWeight: '
        </div>

```

```

    <div style={{ background: 'rgba(112, 187, 142, 0.1)', padding: '1
      <div style={{ fontSize: '11px', color: '#94a3b8' }}>Workouts</d
      <div style={{ fontSize: '20px', color: '#70bb8e', fontWeight: '
    </div>
  </div>
  <div style={{ maxHeight: '300px', overflowY: 'auto' }}>
    {mealTypes.map(type => (currentData.meals[type] || []).length > 0
      <div key={type}>
        <p style={{ color: '#70bb8e', fontSize: '11px', fontWeight: '
          {(currentData.meals[type] || []).map(m => (
            <div key={m.id} style={{ fontSize: '13px', color: '#cbd5e1'
              )))
        </div>
      )}}
    {(currentData.workouts || []).length > 0 && (
      <div>
        <p style={{ color: '#70bb8e', fontSize: '11px', fontWeight: '
          {(currentData.workouts || []).map(w => (
            <div key={w.id} style={{ fontSize: '13px', color: '#cbd5e1'
              )))
        </div>
      )}
    </div>
  </div>
</div>
)}

{/* PROGRESS */}
{activeTab === 'progress' && (
  <div>
    <h2 style={{ marginTop: 0 }}><img alt="📸" data-bbox="515 625 535 635"/> Progress Pictures</h2>
    <button onClick={() => { const url = prompt('Image URL:'); if (url) s
    <div style={{ display: 'grid', gridTemplateColumns: 'repeat(auto-fill
      {Object.entries(progressPictures).map(([date, url]) => (
        <div key={date} className="glass" style={{ padding: '12px' }}>
          <img src={url} alt="Progress" style={{ width: '100%', height: '
          <p style={{ fontSize: '12px', margin: '0 0 8px 0' }}>{new Date(
            <button onClick={() => setProgressPictures(prev => { const n =
          </div>
        )}}
      </div>
    </div>
  )}
</div>
</div>
);
}

```

